

Faster Without Being Wrong

Evaluating Temporal Semantic Caching and Workflow Optimization in Agentic Plan-Execute Pipelines

IBM Team #9

Krish Rakesh Veera · kr2123

Alimurtaza Mustafa Merchant · amm2640

Sajal Kumar Goila · sg4607

Shambhawi Bhure · sb5185

<https://github.com/alimurtaza0411/Latency-Optimized-AssetOpsBench/tree/feature/ablation-study>

Motivation & Problem

Why this matters

- Industrial asset operations workflows are latency-sensitive because a single user query may require coordination over sensor data, work orders, failure modes, forecasting tools, and domain-specific agents.

Problem statement

MCP Workflow Optimizations such as discovery phase caching + parallel step execution and Query Level Optimization such as temporal semantic caching would reduce latency.

Goal:

- 1.5x speedup in MCP Orchestration
- 50% reduction in E2E latency due to query caching

Technical Approach: Optimizations Applied

1. Temporal Semantic Caching

Operators ask the same question about the same asset repeatedly, and if yesterday's answer is still valid today, there's no reason to pay the full pipeline cost again. The dangerous edge case is "yesterday", asked on 2 different days it means 2 different things.

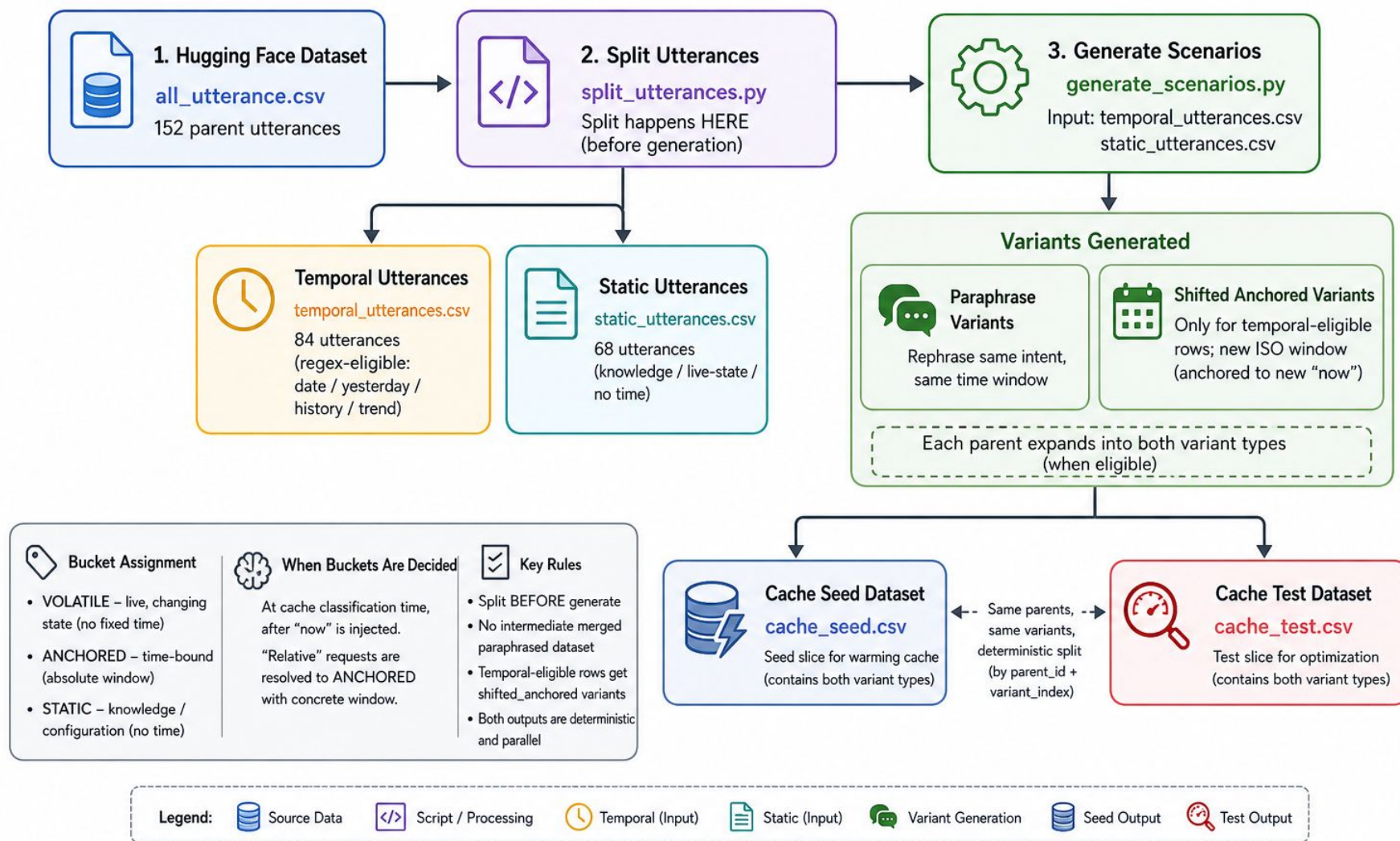
2. MCP Workflow Optimization: Discovery Phase Caching

The tool catalog doesn't change between queries, so paying to spawn 4 server subprocesses and collect signatures on every single query is just wasted overhead you pay once and reuse.

3. MCP Workflow Optimization: Parallel Step Execution with Server Pool

Steps in the plan-execute pipeline which do not depend on each other should not be run sequentially. That would mean each step sits idle waiting for the previous one to finish for no reason. Parallelizing execution steps is useless if they all end up blocked waiting for a single tool server to respond. Keeping a pool of ready server instances ensures that concurrent tool requests actually execute in parallel rather than just waiting in line.

Testing Methodology: Dataset Preparation



Testing Strategy And Profiling Metrics

1. Scenario Selection

Workflow optimization (discovery + parallel) → 20 queries whose plans contain at least 2 independent steps.

Run each query 3× to absorb cloud-LLM variance.

Temporal Semantic cache → seed pass + test pass design.

20 warm parents → cache is pre-populated with their answers

80 test rows → 60% paraphrases of warm parents (cache should hit),

40% paraphrases of held-out cold parents (cache should not hit)

2. How We Measured

Paired comparison: Every query runs twice, once with the optimization off, once on.

Speedup is measured per query, not in aggregate, so cloud latency variance affects both sides equally.

3. Metrics

Latency (both experiments): Median speedup = baseline/cached, computed per query

Cache cost (Temporal Semantic cache): Miss overhead = cached – baseline on misses

Cache quality (Temporal Semantic cache): Precision, Recall and F1

Results: Headline Numbers

Temporal Semantic Cache

30.6x

faster inference (on cache hits)

Both MCP Workflow Optimizations

40.0%

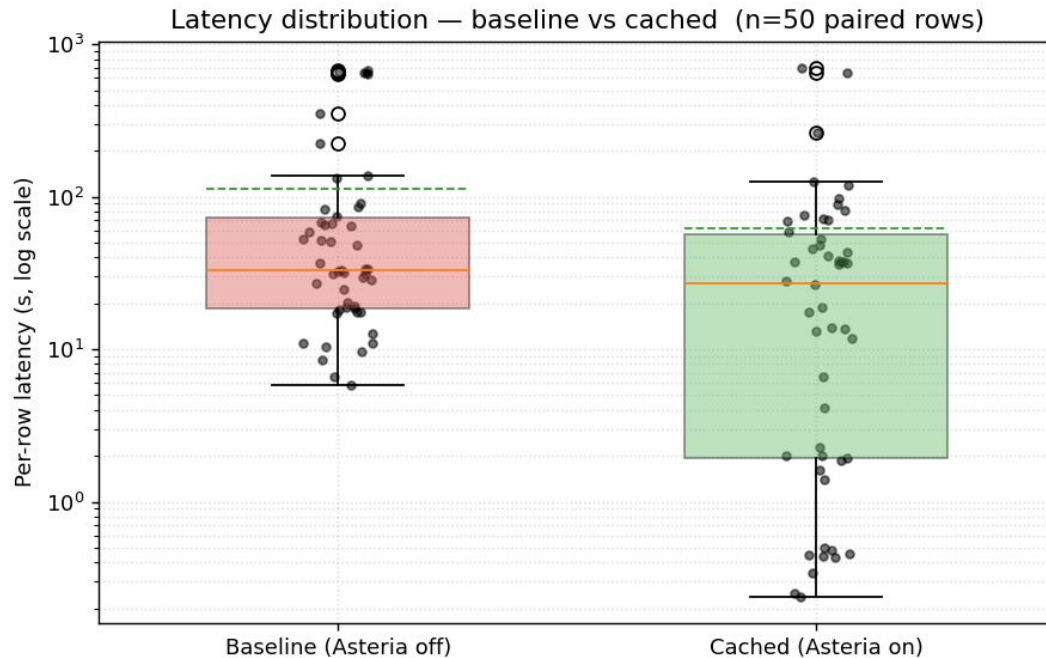
less median end-to-end latency

1.67x

faster responses to queries

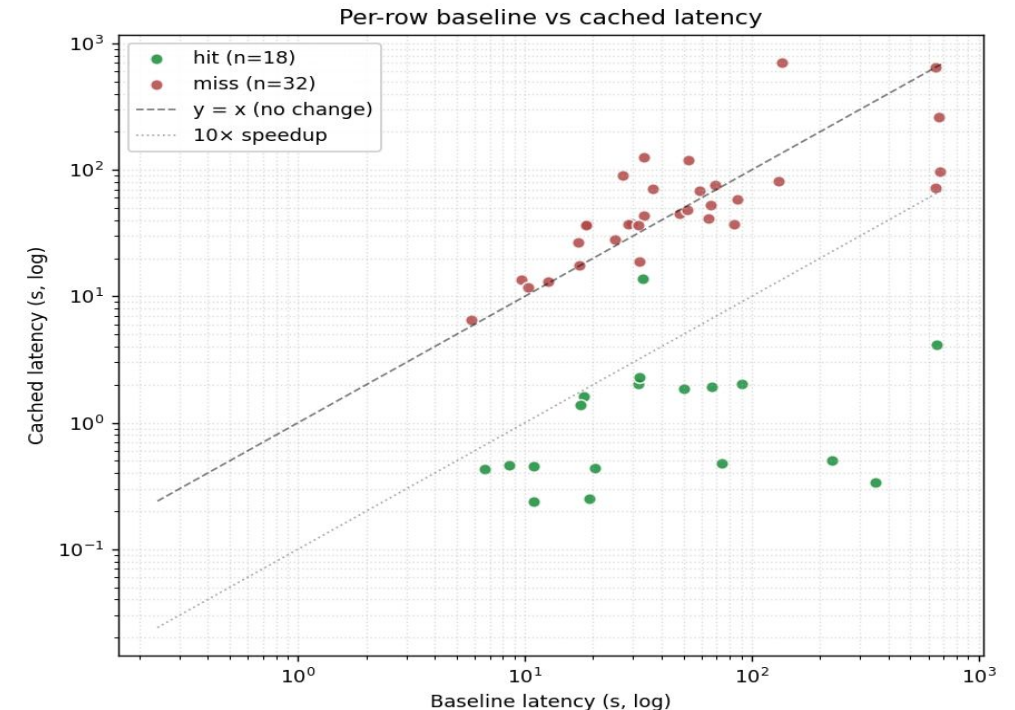
Across 20 benchmark queries with 3 runs per query, MCP workflow optimization reduces median end-to-end latency from 56.90s to 23.02s, corresponding to a 1.67x speedup and a 40.0% latency reduction. In a 50-row temporal-cache benchmark, cache hits achieve a median 30.6x speedup.

Results: Before vs. After (Asteria Only)



Takeaway

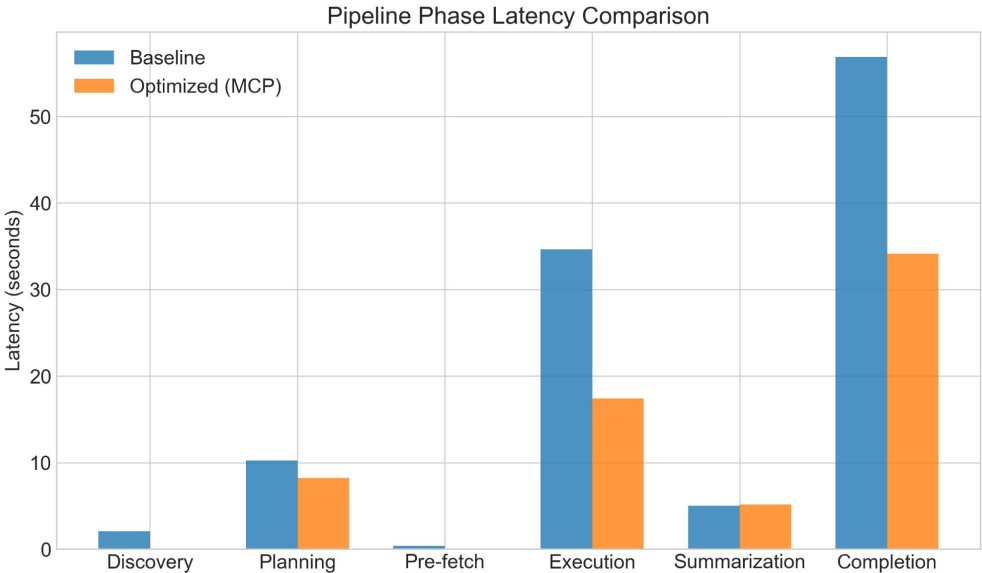
- Box plot of baseline and cached latency distributions across the 50 evaluation rows.
- The cached distribution is compressed at the low end by hit rows, while the miss distribution tracks the baseline with a small added lookup overhead.



Takeaway

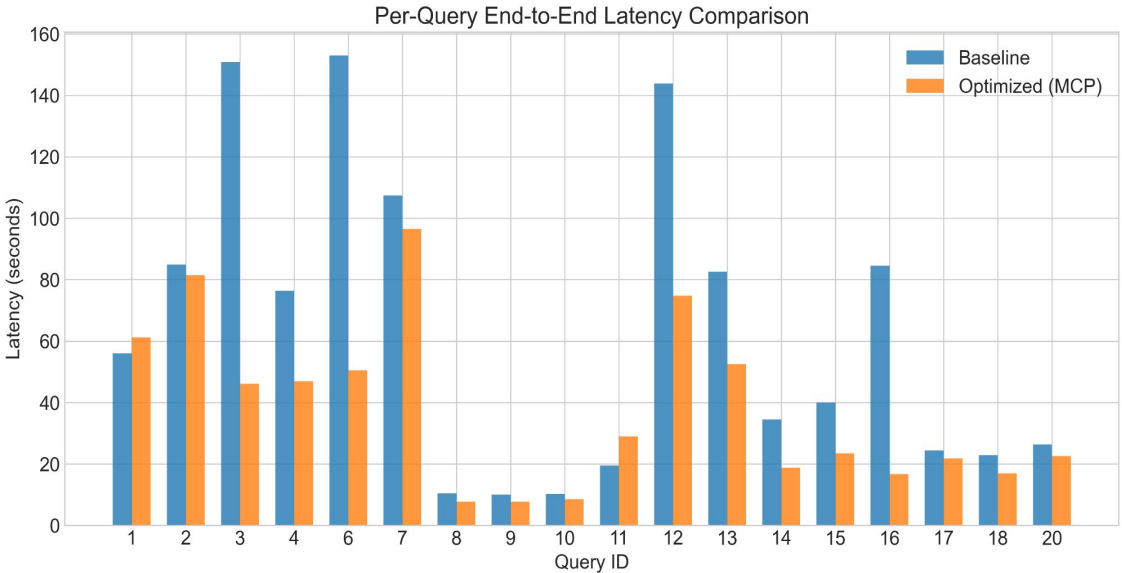
- Per-row scatter of baseline versus cached latency for all 50 evaluation queries. Each point represents one query.
- Points above the diagonal indicate rows where the cached pipeline was slower than the baseline (miss overhead), while points below the diagonal indicate rows where the cache saved time. Cache-hit rows cluster near zero cached latency regardless of baseline cost.

Results: Before vs. After (MCP Optimization Only)



Takeaway

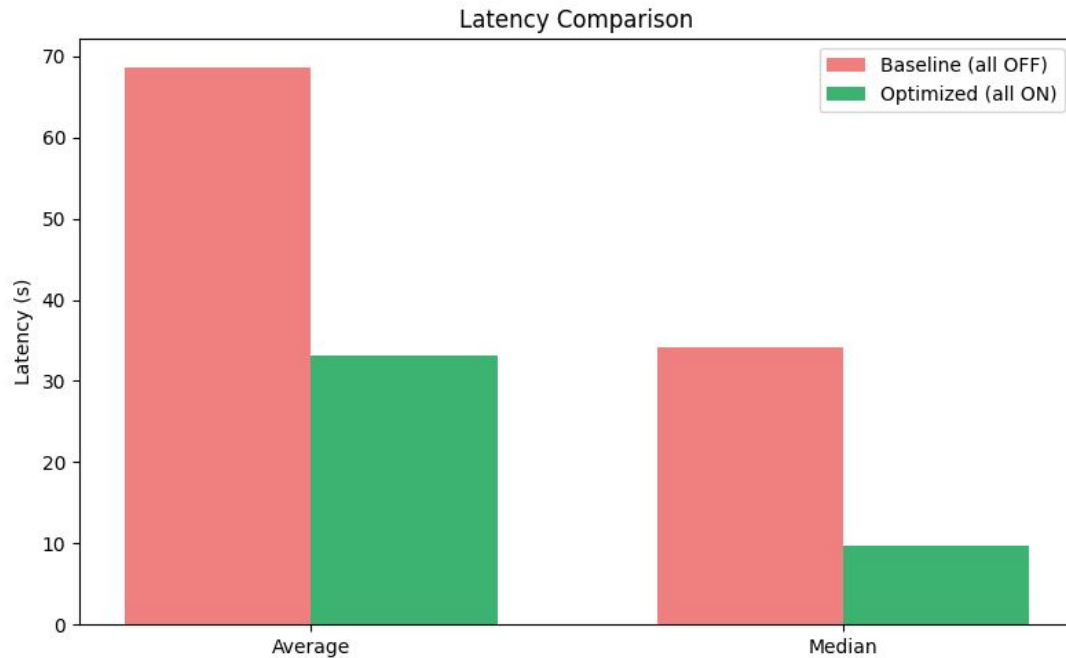
- The MCP-optimized pipeline achieved a massive 40.0% reduction in average end-to-end latency.
- The Discovery phase saw the most dramatic improvement with a 296x speedup, and the pre-fetch phase accelerated by 210x.
- Core execution latency was cut almost in half (1.99x speedup).



Takeaway

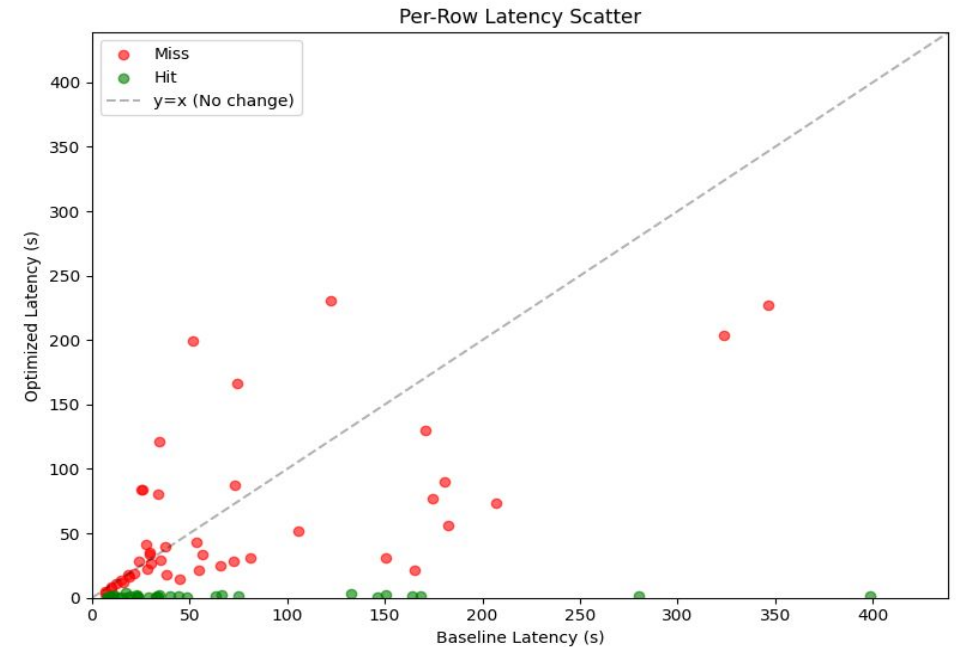
- The vast majority of queries experienced significant latency reductions under the MCP-optimized architecture.
- Complex, multi-step queries saw the highest gains due to improved parallel execution
- For simpler queries, the optimized model remained highly competitive, ensuring no significant performance regressions while scaling up for complex workloads.

Results: Before vs. After (E2E Flow)



Takeaway

- Overall end-to-end latency comparison between the unoptimized baseline and the fully optimized pipeline across all 80 evaluation queries.
- The optimized pipeline reduces median latency from 34.10s to 9.80s, driven primarily by cache hits but with MCP-level gains visible on miss rows as well.



Takeaway

- Per-row latency for all 80 evaluation queries, ordered by row ID.
- Baseline and optimized latencies are shown side by side. Cache-hit rows collapse to near-zero optimized latency, while cache-miss rows show the optimized pipeline tracking below the baseline with MCP-level savings.

Profiler Evidence

A Custom Profiling Script was used to measure wall clock time for net speedup in terms of latency

```
=====
ASSETOPSBENCH BATCH BENCHMARK - SUMMARY REPORT
=====
```

Queries: 20
Runs/query: 3 per mode
Model: watsonx/meta-llama/llama-3-3-70b-instruct
Generated: 2026-04-26 20:26:39

```
=====
```

Phase	Baseline (med)	Optimized (med)	Speedup
Discovery	2.096s	0.007s	296.08x
Planning	10.285s	8.226s	1.25x
Pre-fetch	0.415s	0.002s	210.53x
Execution	34.639s	17.415s	1.99x
Summarization	5.016s	5.165s	0.97x
Completion	56.902s	34.164s	1.67x

Average time saved: 22.738s (40.0% faster)

- **Execution Bypassed:** 95% latency reduction on cache hits.
- **Idle Gaps Eliminated:** 121x mean speedup by avoiding repetitive operations.
- **Bottlenecks Removed:** Discovery and Prefetch latency shrank >200x, driving 40% faster overall completion.

```
=====
Benchmark Summary
=====
```

baseline runs : 50 avg=111.77s trimmed_avg=92.06s med=33.29s min=5.80s max=676.19s
cached runs : 50 avg=61.92s trimmed_avg=37.96s med=27.29s min=0.24s max=703.81s
cache hits : 18/50 (36.0%)

Asteria latency (paired rows) – overall avg mixes hits + misses; use the rows below for end-user benefit.
On cache HITS only (n=18):
speedup baseline/cached mean=121.15x median=30.62x geom_mean=38.82x
time saved vs baseline mean=94.2% median=96.7% (per-row (b-c)/b)
seconds saved (hit rows) mean=93.70s median=24.73s
On cache MISS (n=32) – extra vs baseline:
cached-baseline mean=-25.18s median=+2.23s (lookup + pipeline; expected ≥ 0 often)

```
=====
Benchmark Summary
=====
```

baseline runs : 80 avg=68.68s med=34.10s min=6.73s max=398.73s
cached runs : 80 avg=33.06s med=9.80s min=0.26s max=230.78s
cache hits : 36/80 (45.0%)

Asteria latency (paired rows) – overall avg mixes hits + misses; use the rows below for end-user benefit.
On cache HITS only (n=36):
speedup baseline/cached mean=59.32x median=31.87x geom_mean=33.25x
time saved vs baseline mean=95.0% median=96.9% (per-row (b-c)/b)
seconds saved (hit rows) mean=59.87s median=25.50s
On cache MISS (n=44) – extra vs baseline:
cached-baseline mean=-15.78s median=-3.30s (lookup + pipeline; expected ≥ 0 often)

Demo

Lessons Learned

What worked

- **Discovery cache cut server-spawn overhead** from 2.3s to 8ms - tool signatures never change between queries, unless source code changes.
- **Parallelism gains scaled with plan structure**: more independent branches meant bigger speedup.
- The temporal classifier's **regex approach was fast enough** to run on every query **without eating into the savings** it was trying to preserve.
- VOLATILE bypass worked cleanly: **live-state queries were correctly excluded in microseconds**, preventing the cache from ever serving stale real-time data.
- **Resolving relative phrases** like "yesterday" to concrete ISO windows at insertion time made **cross-day correctness deterministic** rather than relying on the judger to catch it.

What didn't work

- **Parameter-rich queries broke semantic caching** structurally: "Tonnage Chiller 6+9" and "% Loaded Chiller 6" scored 0.97 cosine similarity.
- Judger (**Qwen3-Reranker-0.6B**) **was inconsistent** on legitimate paraphrases, identical intent scored anywhere from 0.5 to 0.95.
- Natural dates like "June 2020" **failed the window extractor and lost temporal pre-filter benefit**.
- Hit rate ceiling on full AOB corpus was 15-30%, **most queries are parameter-rich data fetches**, not knowledge questions.

Next Steps

- **Parameter-aware caching:** cache under *(intent, asset, sensor, time-window)* tuples instead of embedding similarity alone; eliminates the false-positive class entirely.
- **Better judge:** swap Qwen3-Reranker-0.6B for the 4B variant or a domain-adapted reranker to fix the inconsistency on legitimate paraphrases.
- **Richer date parser:** handle "June 2020", "Sept 19 at 7pm" etc. so more queries get proper ANCHORED treatment instead of falling back to STATIC.
- **Cache persistence:** pickle the cache + FAISS index so a process restart doesn't wipe the warm state.
- **Bigger evaluation corpus:** 152 AOB utterances is too small; generate 1000+ with the existing paraphrase generator to tighten all the claims.

Teamwork & Contributions

Member	Primary contributions	Tools / artifacts
Krish Veera	Initial MCP Workflow Optimization Architecture, Temporal Semantic Caching Integration and Debugging, Report Writing	Discovery Caching Script V1, Work Scenario Generator Script V2, Temporal Filter V2
Alimurtaza Merchant	Exploratory Data Analysis and Initial Semantic Caching Architecture, Report Writing	AOB Query Dataset EDA, Semantic Caching Script V1, Temporal Asteria Stress Testing, Report Writing
Sajal Goyla	Parallel Step Execution Script Architecture and Stress Testing, Overall Optimization Integration (Workflow + Query Level) and Stress Testing	Parallel Step Execution Independent Script, Temporal Filter Logic Implementation V1
Shambhawi Bhure	MCP Workflow Optimization Architecture Rewrite (Discovery Caching + Parallel Step Execution) and Stress Testing, Report Writing	Work Scenario Generator Script V1, Collated MCP Workflow Optimization Script

Collaboration: weekly meetings on every weekend, in-person at Uris Library

Thank You

Questions?

GitHub:

<https://github.com/alimurtaza0411/Latency-Optimized-AssetOpsBench/tree/feature/ablation-study>